AIGP Transformation Portal

منصة التحول للذكاء الاصطناعي المساعد — IT Handover Pack

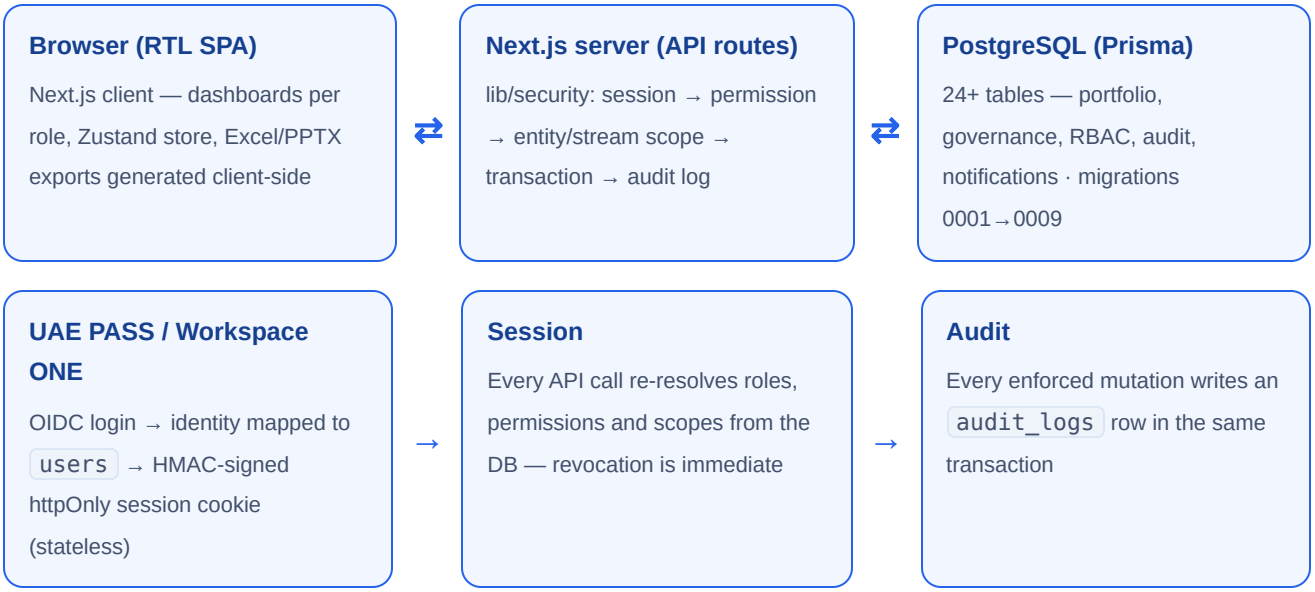
Version: it-handover @ 9a2921e · 09 July 2026

Stack: Next.js 14 (App Router, TypeScript) · PostgreSQL 14+ · Prisma · Docker Compose / K8s

Includes the server-enforced RBAC security layer (roles, permissions, scopes, audit log)

Contents

1 · System architecture & data flow



Data flow (item lifecycle): the entity coordinator (منسق المسار) creates a مشروع/مبادرة/عملية/خدمة as a draft → submits it → the entity representative (ممثل الجهة) approves, rejects, or returns it with notes → approved items enter execution stages (المراحل) with launch plans → stream heads (رؤساء المسارات) nominate items → the national committee (اللجنة الوطنية) reviews nominations and approves funding from the committee wallet. Every step is permission-checked server-side and audit-logged; the client UI is presentation only.

Streams & types: five streams — العمليات والدعم المؤسسي (`ops`), العمل الحكومي الاستراتيجي (`strategy`), الخدمات (`services`), بناء القدرات والتدريب (`capacity`), تقنيات الذكاء الاصطناعي والبيانات (`tech`). مشروع/مبادرة exists in all streams; عملية only in `ops` + `strategy`; خدمة only in `services`. The UI enforces this scoping everywhere (filters, tabs, KPIs, creation wizard).

Everything the IT team needs to stand up the production deployment: database, roles, authentication hooks, and operational notes. The repository is a Next.js 14 (App Router, TypeScript) application with a PostgreSQL schema managed by Prisma.

1. Runtime & prerequisites

Component	Requirement
Node.js	20 LTS or newer
PostgreSQL	**14+ recommended** (12 minimum — migration `0006` uses `ALTER TYPE … ADD VALUE` inside a transaction)
Package manager	npm (lockfile committed)

```
cp .env.example .env         # then fill in the (REQUIRED) values — see §2
npm ci
npm run db:setup             # migrate deploy (0001 → 0009) + generate + seed
npm run build && npm start   # serves on PORT (default 3000)
```

Or, fully containerised (Postgres + app, migrations + seed on first boot):

```
cp .env.example .env         # fill in the (REQUIRED) values
docker compose up --build    # app on :3000, Postgres on :5432
```

The seed loads **reference data and starter accounts only** — streams, entities, phases, stages, roles/permissions, and the placeholder users. The portfolio starts **empty**; set `SEED_DEMO_ITEMS=1` only on a demo/staging database if sample items are wanted.

What IT changes: only the values in `.env` (DB URL, `SESSION_SECRET`, UAE PASS client id/secret/redirect, optional AI endpoint). No code edits are required. `NEXT_PUBLIC_*` values are build-time — set them before the build.

2. Environment variables

The single source of truth is `.env.example` — every variable is listed there with `(REQUIRED)` / `(build-time)` markers and inline guidance. Copy it to `.env` and fill in the blanks. Summary:

Variable	Purpose
`DATABASE_URL`	Postgres connection string (REQUIRED)
`NEXT_PUBLIC_DATA_MODE`	`api` in production (build-time); `local` = static demo
`SESSION_SECRET`	Signs the stateless session cookie — `openssl rand -hex 32` (REQUIRED)
`SESSION_TTL_HOURS`	Session lifetime (default 12; `SESSION_TTL_SECONDS` overrides)
`AUTH_PROVIDER` / `NEXT_PUBLIC_AUTH_PROVIDER`	`mock` (dev only) \ `uaepass` \ `workspaceone`
`BOOTSTRAP_ADMIN_EMAILS`	Auto-provision these emails as `system_admin` on first login
`OIDC_ISSUER` / `_CLIENT_ID` / `_CLIENT_SECRET` / `_REDIRECT_URI`	Workspace ONE OIDC (REQUIRED for `workspaceone`)
`STATE_API_TOKEN`	Bearer guard for `/api/state` on shared deployments
`NEXT_PUBLIC_UAEPASS_MODE`	`live` in production; `mock` for the demo (build-time)
`UAEPASS_CLIENT_ID` / `_SECRET` / `_REDIRECT_URI`	UAE PASS OIDC — see §5 (REQUIRED for live)
`AI_API_BASE_URL` / `_KEY` / `_MODEL`	Internal AI reviewer endpoint (optional)
`NEXT_PUBLIC_BASE_PATH`	Only for sub-path hosting (build-time)
`NEXT_PUBLIC_DEMO_MODE` / `_DATA`	**Keep `0` in production** — role switcher / mock data

Secrets belong in the host's secret manager — never in the repository. `NEXT_PUBLIC_*` variables are inlined at build time, so set them before `npm run build` / the Docker image build.

3. Database structure (Prisma → Postgres)

Schema: `prisma/schema.prisma` (tables/columns are snake_case via `@map`; the client uses camelCase). Migrations: `prisma/migrations/0001…0009`.

Reference data

- `streams` — the five transformation streams (ids: `ops`, `strategy`, `services`, `capacity`, `tech`) with the official stream heads.
- `entities` — federal entities (seeded with the 34 entities).
- `program_phases` — program phases with editable deadlines (committee).
- `exec_batches` — the four execution/launch stages (المراحل) with periods.
- `settings` — key/value (e.g. `approvedBudget` for the committee wallet).

Team setup (per entity)

- `entity_reps` — official entity representative (one per entity).
- `stream_owners` — per-stream owner inside the entity (unique per entity+stream).

Portfolio

- `items` — every مشروع/مبادرة/عملية/خدمة. Carries the assessment fields, outcome/scope fields, type-specific fields, the execution stage (`exec_batch`), the returned-with-notes state (`ret_`), *and the stage-move marker* (`stage_move_` — set when an item is moved between stages; the client notifies all stakeholders). `wf` is the workflow enum: `draft → ent1 → pm1 → pm2 → ent2 → budget → exec → launch → done` (the UI presents the simplified delivery view: قيد التطوير / تم التطوير / تم الإطلاق on top of `budget|exec / launch / done`).
- `launch_plans` — centrally managed launches per stage (name, type, date, launch-level scope, informational launch budget). The **execution budget is derived**: the server/client recomputes it as the sum of attached items' budgets — do not write it independently.
- `item_launch_plans` — join table (an item may join several launches, all within its single stage).
- `exec_checklist_items`, `sub_milestones` — per-item execution details.
- `launches`, `item_launches` — legacy per-item launch rows kept for the detail view history; new planning goes through `launch_plans`.

Governance

- `log_entries` — the approval/action audit trail shown in السجل.
- `nominations` — ترشيحات رؤساء المسارات (what the committee reviews).
- `fundings` / `funding_cancellations` — committee funding decisions with the mandatory cancellation reason.

Access control (RBAC), audit & notifications (migrations `0007` – `0009`)

- `roles` — backend access roles keyed by a stable `code` (nine codes, see §4). `users.role` keeps the *legacy UI key* (`admin`, `coord`, `entity`, `path`, `ai`) as a plain string for the client screens; server enforcement goes through `user_roles` / `role_permissions`.
- `permissions` — stable permission codes (`items:approve`, `funding:cancel`, …).
- `role_permissions` — role → permission matrix (seeded, editable).
- `user_roles` — role assignment per user (the app enforces one role per user).
- `user_entity_scopes` / `user_stream_scopes` — per-user data scopes; the API filters every item query by them (`lib/security/rbac.ts`).

- `audit_logs` — server-side audit trail written inside the same transaction as every enforced mutation (actor, action, resource, entity/stream, IP, user-agent, metadata).
- `role_assignment_rules` — pre-configured email→role(+scopes) mappings; applied automatically on the user's first login (team setup and `/api/admin/role-rules` create them).
- `sessions` — **removed** (migration `0009`). Sessions are now stateless HMAC-signed httpOnly cookies (`lib/security/session.ts`), signed with `SESSION_SECRET`; nothing is stored server-side.
- `notifications` — persisted الإشعارات, targeted to a user or broadcast to a role / entity / stream, with `kind`, title/body, and `read_at`. The client currently derives notifications from data state; this table lets IT persist and push them (email/SMS) from the API layer.

Demo sync

- `app_state` — single JSON blob used by `/api/state` for the demo persistence. Harmless in production; can stay empty.

The full table list (with columns) lives in `prisma/schema.prisma`; the versioned DDL is in `prisma/migrations/0001…0009`.

4. Role management

Access is enforced server-side: nine backend role codes (in `roles`) carry the permission matrix; the client keeps rendering its four UI roles + the admin console via the legacy `users.role` key. Mapping:

Backend code	Arabic	UI role	Scope enforced by the API
`system_admin`	مدير النظام	`admin`	global (bypasses permission checks)
`program_admin`	مدير البرنامج	`ai`	global
`ai_committee`	اللجنة الوطنية	`ai`	global, approved items only
`stream_owner`	رئيس المسار	`path`	own stream across all entities
`entity_representative`	ممثل الجهة	`entity`	own entity across all streams, no drafts
`entity_admin`	مسؤول الجهة	`entity`	own entity + entity updates
`entity_coordinator`	منسق المسار في الجهة	`coord`	own entity + own stream, incl. drafts
`viewer`	مستعرض	`entity`	read-only within scopes
`auditor`	مدقق	`entity`	read-only + `audit:view`

Role → permission assignments live in `role_permissions` (seeded from `prisma/seed.ts`). **Deliberate deviation from the reference permission matrix:** `entity_representative` is granted `items:approve` and `items:reject` — in our confirmed business flow the entity representative is the sole approver at the `ent1` gate (the upstream reference seed omitted these, contradicting its own client behaviour).

Who provisions whom: `system_admin` manages all accounts and role rules (`/api/admin/users`, `/api/admin/role-rules`) and assigns the stream heads (`stream_owner`) and national committee (`ai_committee`); the entity rep provisions its own coordinators in team setup (`/api/team/register` creates `role_assignment_rules` consumed on the coordinator's first login). `BOOTSTRAP_ADMIN_EMAILS` auto-provisions the very first system admin(s) on login.

Seeded starter accounts (in `users`, keyed by email on the `@aigp.gov.ae` placeholder domain): the system admin (`admin@…`), the national committee, the five stream heads (`head.<stream>@…`), the default entity representative (`rep@…`), and one coordinator per stream (`coord.<stream>@…`). Each is seeded active with its backend role in `user_roles` plus the matching entity/stream scopes. To go live, re-point each account's `email` to the verified UAE PASS identity (or deactivate and create real ones).

Rules the application assumes (enforce when provisioning users):

- `coord` and `entity` **must** have `entity_id`; `coord` and `path` **must** have `stream_id`; `ai` has neither.
- One `entity` user per entity is the approver (mirrored in `entity_reps`).
- The five `path` users are the official stream heads (names pre-seeded on `streams.head_name`); attach their emails when known.
- Deactivate (never delete) users via `is_active = false` so the audit log keeps valid author names.

Data visibility implemented by the app (for reference):

- Drafts (`wf = 'draft'`) are visible **only** to the coordinator.
- `ent1` (awaiting entity approval) is visible to coord + entity rep.
- The committee acts only on nominations (`nominations`), not raw items.

5. Server API (enforced endpoints)

Every route below runs `requireAuthUser` → `assertPermission` → `assertItemAccess` (entity/stream scope), mutates inside a transaction and writes an `audit_logs` row (`lib/security/`):

- **Auth**: `GET /api/auth/login` (provider dispatch: mock / uaepass / workspaceone), `POST /api/auth/logout`, `GET /api/auth/me` (roles + permissions + scopes), `GET /callback` (Workspace ONE OIDC redirect), `GET /api/auth/uaepass/login|callback` (UAE PASS, §6).
- **Items**: `GET|POST /api/items`, `GET|PATCH|DELETE /api/items/:id`, `POST /api/items/:id/submit|approve|reject|return` (workflow actions with log entries + audit).
- **Portfolio reads**: `GET /api/funding`, `GET /api/nominations`, `GET /api/launch-plans` — all scope-filtered.
- **Team setup**: `POST /api/team/register` — upserts `entity_reps` / `stream_owners` and creates `role_assignment_rules` for the team.
- **Admin**: `GET|POST /api/admin/users`, `GET|PATCH /api/admin/users/:id`, `POST /api/admin/users/:id/enable|disable`, `POST /api/admin/users/:id/roles` (+ `DELETE …/roles/:roleId`), `GET /api/admin/roles|permissions|entities|streams`, `GET|POST|DELETE /api/admin/role-rules`, `GET /api/admin/audit-logs`.
- **Ops**: `GET /api/health` (liveness), `GET /api/ready` (DB probe).
- **State blob (enforced)**: `GET|PUT /api/state` — requires a signed session AND a global-scope role (`canAccessAllEntities`); scoped users get `{data:null, scoped:true}`. `PUT` additionally honors `STATE_API_TOKEN` when set. Because mock login is disabled with `NODE_ENV=production`, shared server state in production requires a real `AUTH_PROVIDER` (uaepass/workspaceone); until then each browser falls back to local storage.
- **AI reviewer (enforced)**: `POST /api/ai-review` — requires session + `ai_review:run` permission, honors `AI_REVIEW_ENABLED=false` as a kill-switch, and writes an audit log. Clients without access fall back to the built-in heuristic review.

6. Authentication (UAE PASS)

`app/api/auth/uaepass/login` and `…/callback` are fully wired: the callback validates state, exchanges the code, maps the verified identity to `users` via `ensureUserFromIdentity` (bootstrap admins + role-assignment rules, pending/disabled users are turned away), and issues the signed session cookie. The demo build bypasses this flow behind the "Sign in with UAE PASS" button. To go live:

- Register the redirect URI with UAE PASS and set the client id/secret in the environment (`UAEPASS_*` variables), plus `AUTH_PROVIDER=uaepass` and `NEXT_PUBLIC_AUTH_PROVIDER=uaepass` (build arg).
- Nothing else — user mapping and access gating are implemented in `lib/security/user-access.ts`.
- Sessions are stateless cookies (HMAC-signed with `SESSION_SECRET`, `maxAge = SESSION_TTL_HOURS`, httpOnly) via `lib/security/session.ts`; there is no sessions table. `/api/auth/me` resolves `{role, entityId, streamId, name}` plus backend roles/permissions — the client reads the role from it — the role-switcher tabs only exist when `NEXT_PUBLIC_DEMO_MODE=1`.

Invitation emails for entity reps and coordinators are drafted in `docs/email-templates.md` — wire them to the mail gateway when accounts are provisioned.

7. Operational notes

- **Budgets**: item budgets are free-text in Arabic; the parsed numeric value is mirrored into `budget_amount` (BigInt, drhm) for reporting. Plan execution budgets are always recomputed from attached items.
- **Notifications** are currently derived in the client from data state (returns, nominations, funding, stage moves). If IT wants push/email later, the same triggers can be raised from the API layer; templates are in `docs/email-templates.md`.
- **Exports** (Excel/PowerPoint) are generated client-side; no server dependency.
- **Backups**: standard Postgres dumps; all state is in the tables above (nothing critical lives in `app_state`).
- **Security**: run behind the government gateway/WAF; the app sets no cookies of its own in demo mode; rotate any tokens used during the handover (including the temporary GitHub deploy token used for the demo site).

9 · Go-live checklist

	Step	Where
☐	Provision PostgreSQL 14+ and set `DATABASE_URL`	`.env`
☐	Generate `SESSION_SECRET` (`openssl rand -hex 32`) and store it in the secret manager	`.env`
☐	Set `AUTH_PROVIDER=uaepass` and `NEXT_PUBLIC_AUTH_PROVIDER=uaepass` (build-time)	`.env` + compose build args
☐	Register the UAE PASS redirect URI; set `UAEPASS_CLIENT_ID/_SECRET/_REDIRECT_URI`	`.env`
☐	Set `BOOTSTRAP_ADMIN_EMAILS` to the first system admin(s)	`.env`
☐	Keep `NEXT_PUBLIC_DEMO_MODE=0` and `NEXT_PUBLIC_DEMO_DATA=0`	compose build args
☐	`docker compose up --build` — migrations 0001→0009 + seed run automatically	server
☐	Verify `GET /api/health` and `GET /api/ready` return OK	gateway probe
☐	Log in as a bootstrap admin; re-point the seeded starter accounts to real UAE PASS identities (or deactivate them)	admin APIs / DB
☐	Assign stream heads and the national committee; entity reps register their teams (coordinators)	admin APIs / team setup
☐	Optional: set `AI_API_BASE_URL/_KEY/_MODEL` for the AI reviewer (or leave unset — heuristic fallback)	`.env`
☐	Front the app with the government gateway/WAF; enable Postgres backups	infra
☐	Rotate every credential used during the handover (including the temporary GitHub deploy token)	security

Definition of done: a real user logs in through UAE PASS, sees only their scoped data, an approval writes an `audit_logs` row, and `/api/state` refuses unauthenticated calls.

Companion documents in the repository: `docs/it-handover.md` (this content, maintained in git), `docs/Architecture-and-DataFlow.pdf`, `docs/Roles-and-Access.pdf`, `docs/email-templates.md`, `HANDOVER.md`, `DEPLOYMENT.md`, `SECURITY.md`, `k8s/` manifests.